

Streams och strängar

Det finns några problem som relativt ofta förekommer då man gör program som hanterar strängar i olika former. Dessa är bl.a. hur man enkelt skapar strängar av olika andra primitiva datatyper och hur man läser data som finns i strängar och konverterar detta till någonting annat. Dessa båda problemen kan lösas med hjälp av *streams till/från strängar*.

Då vi hittills använt oss av streams har de alltid varit förknippade med skärmen, tangentbordet eller filer (se [Kapitel 19](#)), men aldrig med strängar. Det finns dock någonting som kallas *strängstreams* som vi kan använda för att läsa och skriva strängar.

Skriva till strängar

Vi kan tänka oss ett program som hanterar en SQL-databas och behöver skapa SQL-satser som används för att manipulera data i databasen. SQL-satser representeras normalt som strängar, och skickas sedan till databasen för exekvering, och eventuella resultat returneras på något databas-specifikt sätt. Vi kan tänka oss ett program som skall söka personer i ett företags personregister på bas av arbetsuppgift och lön. Vi vill skapa t.ex. följande SQL-sats:

```
SELECT *
FROM Persons
WHERE Job = 'Hacker' AND Salary > 20000;
```

Om man alltid vill exekvera exakt samma sats har vi inga problem, det är bara att skriva SQL-satsen i en sträng och saken är klar. Vill vi dock kunna ändra på värdena på `Job` och `Salary` under programmets körning måste vi skapa SQL-satsen dynamiskt. Här kan vi använda oss av klassen `ostrstream`. Denna tillåter att vi skriver data till en instans av klassen med `<<` precis som om vi skrev ut data till skärmen. Sedan kan man "plocka ut" den resulterade strängen med metoden `str()`. Ett exempel på hur ovannämnda SQL-sats kan konstrueras med hjälp av `ostrstream`:

```
#include <strstream>
#include <string>

int main () {
    int Salary;
    string Job;

    // läs in värden på lön och jobb
    cout << "Ge jobbtyp: ";
    cin >> Job;
    cout << "Ge lön:      ";
    cin >> Salary;

    // skapa SQL-satsen
    ostrstream SQL;

    SQL << "SELECT *" << "FROM PERSONS " << endl
        << "WHERE Job = '" << Job << "' AND Salary > "
        << Salary << "';";

    // skriv ut den nya satsen
    cout << endl << "SQL-satsen är nu: " << endl
```

```

        << SQL.str () << endl;
    }

```

Vi skapar variabeln `SQL` som en instans av `ostream`. Sedan skriver vi vår önskade sträng precis som om vi skrev till `cout`. Vi kan skriva ut strängar, tal, tecken o.s.v., `ostream` kan konvertera alla datatyper till lämpliga strängar.

För att sedan få tillgång till den resulterade strängen använder vi metoden `str()` som returnerar en sträng, samt skriver ut denna. I ett större program skulle strängen t.ex. skickas till en databas för exekvering. Kör vi programmet ovan får vi t.ex. följande utskrift:

```

% ./StringStream1
Ge jobbtyp: Hacker
Ge lön:      20000

SQL-satsen är nu:
SELECT *FROM PERSONS
WHERE Job = 'Hacker' AND Salary > 20000;
%

```

Notera att detta är ett enkelt sätt att konvertera tal till motsvarande sträng! Ett exempel:

```

#include <sstream>
#include <string>

int main () {
    float Tal;

    // läs in ett tal
    cout << "Ge flyttal: ";
    cin >> Tal;

    // skapa stream
    ostream Stream;

    // utför konverteringen
    Stream << Tal;

    // hämta det konverterade talet
    string TalStrang = Stream.str ();

    cout << "Talet som sträng: " << TalStrang << endl;
}

```

Programmets utskrift kan se ut på följande sätt:

```

% ./StringStream2
Ge flyttal: 3.045
Talet som sträng: 3.045
%

```

Notera att den headerfil som inkluderas är `<sstream>`, men enligt standarden för C++ borde filen heta `<sstream>`. Detta är ett resultat av att inte alla system ännu fullt följer standarden. Se [avsnittet *Strängstreams och kompatibilitet*](#) för mera information.

Läsa från strängar

Motsatsen till det ovannämnda kan även göras, d.v.s. man kan läsa data ur en sträng precis på samma sätt som man kan läsa från `cin`. Detta gör det lättare att t.ex. skriva en parser som tolkar data ur en sträng. Klassen som används heter då `istream`. Vi kan skapa ett enkelt program som läser en rad med text och delar upp den i dess beståndsdelar, d.v.s. ord, på följande sätt:

```
#include <sstream>
#include <string>

int main () {
    string Rad;
    string Ord;
    int Antal = 1;

    // läs in en rad
    cout << "Ge en rad med text: ";
    getline ( cin, Rad );

    // skapa stream
    istringstream Stream ( Rad.c_str () );

    // iterera så länge det finns flera ord
    while ( Stream >> Ord ) {
        cout << "Ord " << Antal++ << " = " << Ord << endl;
    }
}
```

Vi använder oss här av funktionen `getline()` för att läsa en hel rad med data från `cin` in i `Rad`. Mera information om denna funktion finns i [avsnittet Läs radvis i Kapitel 19](#). Normala `cin >> Rad;` skulle inte gett önskat resultat, eftersom den endast läser ett ord åt gången. Därefter skapas en `istringstream` av den inlästa raden. Ett problem här är att den konstruktor som finns tillgänglig på det använda systemet inte accepterar `string`, utan kräver C-strängar, varvid metoden `c_str()` används för att konvertera till en sådan. Läs mera på [avsnittet Konvertera till C-strängar](#). Efter detta är det sedan bara att läsa ord för ord ur den inlästa strängen så länge det finns data.

Denna klass kan även användas för att läsa andra datatyper än endast `string`. Vi kan således enkelt kontrollera om en inläst sträng är t.ex. ett giltigt tal eller inte med följande:

```
#include <sstream>
#include <string>

int main () {
    string Text;
    string Rest;
    int Tal;

    // läs in en rad
    cout << "Ge ett tal: ";
    cin >> Text;

    // skapa stream
    istringstream Stream ( Text.c_str () );

    // läs ett tal om det går
    if ( Stream >> Tal ) {
        // talet är ok
        cout << "Läste talet " << Tal << endl;

        // finns det mera text kvar i buffern?
        if ( Stream >> Rest ) {
            // jep, mera text kvar, så det är kanske inte ett ok tal?
        }
    }
}
```

```

        cout << "Texten '" << Rest << "'" blev oläst." << endl;
    }
}
else {
    // inget giltigt tal
    cout << "Texten '" << Text << "'" är inte ett tal." << endl;
}
}

```

Programmet läser ett tal och kontrollerar ifall det kunde konverteras till ett heltal. Notera dock att `istream` endast läser tecken ur strängen så länge som de passar in med den datatyp som läses. Om vi ger som tal texten `0xffff` i tron att den läser ett hexadecimalt tal kommer den endast att läsa den första nollan och ignorera resten. Därför har vi en extra `if`-sats för att kontrollera ifall det blev text kvar i strängen. Om det finns text kvar kunde inte hela ordet konverteras och det är troligtvis felaktigt. Samma händer om vi t.ex. försöker läsa strängen `10.73` in i ett heltal; allting efter nollan ignoreras. Körning av programmet kan t.ex. ge:

```

% ./StringStream4
Ge ett tal: 54
Läste talet 54
% ./StringStream4
Ge ett tal: 0xffff
Läste talet 0
Texten 'xffff' blev oläst.
%

```

På detta sätt kan man bygga upp robust inläsning av data från tangentbord och filer. Det lönar sig alltid att kontrollera giltighet på inläst data!

Strängstreams och kompatibilitet

Som du märkt är det ännu vissa problem med system som inte fullt följer standarden för C++. Detta manifesteras ganska tydligt då man använder sig av sträng-streams. De klasser som används i exemplen ovan, d.v.s. `istream` och `ostream` heter egentligen `istreamstream` och `ostreamstream`, men dessa klasser finns inte på det system som denna kurs i huvudsak använder sig av. Analogt så skall man egengligen inkludera headerfilen `<sstream>` istället för `<sstream>` som vi gjort i exemplen. Om det system du använder följer standarden bättre skall du använda dessa filer och klasser istället.

I en senare version av detta kompendium kommer de riktiga klasserna och headerfilerna att användas.

[Förutgående](#)
Substrängar

[Hem](#)
[Upp](#)

[Nästa](#)
Manipulera tecken